

```
In [15]: # Load CSV file
import pandas as pd

df = pd.read_csv('Sales_2014.csv')
```

Task 1: Data Preparation

- Load and inspect dataset (first 5 rows, data types, missing values).
- Clean data: convert dates to datetime, handle missing values, create Total Sales.

```
In [18]: # First 5 rows
df.head(5)
```

```
Out[18]:
```

	Order ID	Customer ID	SalesPerson	Order Date	Order Priority	SKU	Order Quantity	Unit Sell Price	Discount	Shipping Amount	Ship Mode	Product Container	Ship Date
0	13729	CS08	Bob	1/1/2014	Not Specified	SKU047	9	95.99	0.08	35.00	Express Air	Large Box	1/5/2014
1	28774	C372	John	1/1/2014	High	SKU037	32	5.98	0.10	4.69	Regular Air	Small Box	1/2/2014
2	9085	C212	Bob	1/2/2014	Critical	SKU363	3	40.98	0.06	2.99	Regular Air	Small Box	1/4/2014
3	37637	C015	Bob	1/2/2014	Low	SKU052	4	291.73	0.00	48.80	Delivery Truck	Jumbo Drum	1/2/2014
4	37637	C015	Bob	1/2/2014	Low	SKU063	43	100.98	0.07	45.00	Delivery Truck	Jumbo Drum	1/4/2014

```
In [19]: df.dtypes
```

```
Out[19]:
```

Order ID	int64
Customer ID	object
SalesPerson	object
Order Date	object
Order Priority	object
SKU	object
Order Quantity	int64
Unit Sell Price	float64
Discount	float64
Shipping Amount	float64
Ship Mode	object
Product Container	object
Ship Date	object
dtype:	object

```
In [21]: # Convert Order Date and Ship Date to datetime format
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

df.dtypes
```

```
Out[21]:
```

Order ID	int64
Customer ID	object
SalesPerson	object
Order Date	datetime64[ns]
Order Priority	object
SKU	object
Order Quantity	int64
Unit Sell Price	float64
Discount	float64
Shipping Amount	float64
Ship Mode	object
Product Container	object
Ship Date	datetime64[ns]
dtype:	object

```
In [24]: # Create Total Sales column
df['Total Sales'] = df['Order Quantity'] * df['Unit Sell Price'] * (1 - df['Discount'])
df[['Order Date', 'Ship Date', 'Total Sales']].head()
```

```
Out[24]:
```

	Order Date	Ship Date	Total Sales
0	2014-01-01	2014-01-03	794.7972
1	2014-01-01	2014-01-02	172.2240
2	2014-01-02	2014-01-04	115.5636
3	2014-01-02	2014-01-02	1169.9200
4	2014-01-02	2014-01-04	4038.1902

```
In [18]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2153 entries, 0 to 2152
Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  --
0   Order ID            2153 non-null   int64
1   Customer ID         2153 non-null   object
2   SalesPerson         2153 non-null   object
3   Order Date          2153 non-null   datetime64[ns]
4   Order Priority       2153 non-null   object
5   SKU                 2153 non-null   object
6   Order Quantity      2153 non-null   int64
7   Unit Sell Price     2153 non-null   float64
8   Discount            2153 non-null   float64
9   Shipping Amount     2153 non-null   float64
10  Ship Mode           2153 non-null   object
11  Product Container   2153 non-null   object
12  Ship Date           2153 non-null   datetime64[ns]
13  Total Sales         2153 non-null   float64
dtypes: datetime64[ns](12), float64(8), int64(1), object(6)
memory usage: 235.6+ KB
```

Task 2: Salesperson Performance

- Plot total sales by salesperson (bar plot).
- Recommend a bonus recipient, considering total sales and another factor (e.g., order count). Propose a new metric and justify in 2-3 sentences.

```
In [20]: # Group and Sum Total Sales by Salesperson in one dataframe
sales_by_person = df.groupby('SalesPerson')['Total Sales'].sum().sort_values(ascending=False)
sales_df = sales_by_person.reset_index()
sales_df[['Total Sales']] = sales_df[['Total Sales']].round(10).astype(int).apply(lambda x: f'{x:,}')
sales_df
```

```
Out[20]:
```

	SalesPerson	Total Sales
0	Richard	1,542,980
1	Bob	1,373,385
2	John	1,232,242

```
In [22]: # Bar Plot
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=sales_by_person.index, y=sales_by_person.values, palette='viridis')

for i, value in enumerate(sales_by_person.values):
    ax.text(value - 0.5e5, i, f'{int(value):,}', va='center', ha='right', color='yellow', fontsize=10, fontweight='bold')
```

```
plt.xlabel('Total Sales')
plt.ylabel('Salesperson')
plt.title('Total Sales by Salesperson')
plt.tight_layout()
plt.show()
```



```
In [24]: # Create numpy import
import numpy as np
# Total sales, order count by salesperson
sales_summary = df.groupby('SalesPerson').agg(['Total Sales', 'sum', 'Order ID', 'count']).rename(columns={'Order ID': 'Order Count'})

# Sales Score = Total Sales * log(Order Count + 1) - new metric
sales_summary['Sales Score'] = sales_summary['Total Sales'] * np.log(sales_summary['Order Count'])

# Change data types & format
sales_summary['Total Sales'] = sales_summary['Total Sales'].round(0).astype(int)
sales_summary['Sales Score'] = sales_summary['Sales Score'].round(0).astype(int)
sales_summary['Order Count'] = sales_summary['Order Count'].astype(int)
sales_summary = sales_summary.sort_values(by='Sales Score', ascending=False)
sales_summary
```

```
Out[24]:
```

	Total Sales	Order Count	Sales Score	
0	Richard	1542980	729	10172955
1	Bob	1373385	730	9056667
2	John	1232242	694	8065890

Bonus Recommendation: Bonus should go to Richard. Richard generated the highest total sales and handled a very high number of orders as well.

Task 3: Sales Trends

- Plot monthly total sales (line plot).
- Suggest one 2015 marketing strategy based on trends. Justify in 2-3 sentences with specific months and sales.

```
In [26]: import matplotlib.pyplot as plt
import seaborn as sns

df['Year-Month'] = df['Order Date'].dt.to_period('M')
monthly_sales = df.groupby('Year-Month')['Total Sales'].sum().reset_index()
monthly_sales['Year-Month'] = monthly_sales['Year-Month'].astype(str)
print(f'Uniquely Total Sales:')
monthly_sales
```

```
Out[26]:
```

	Year-Month	Total Sales
0	2014-01	603479.4488
1	2014-02	330449.9852
2	2014-03	402062.6992
3	2014-04	385175.1344
4	2014-05	228989.6496
5	2014-06	255981.4546
6	2014-07	378143.5598
7	2014-08	332645.4162
8	2014-09	324590.9374
9	2014-10	355178.3955
10	2014-11	245342.3499
11	2014-12	406688.1061

```
In [28]: # Line plot
plt.figure(figsize=(10, 6))
sns.lineplot(x='Year-Month', y='Total Sales', data=monthly_sales, marker='o', color='blue')

for i in range(len(monthly_sales)):
    plt.text(
        x=monthly_sales['Year-Month'][i],
        y=monthly_sales['Total Sales'][i] + 5000,
        s=f'Int(monthly_sales["Total Sales"][{i}]),',
        ha='center',
        va='bottom',
        fontsize=10,
        color='black')

plt.xlabel('Year-Month')
plt.ylabel('Total Sales')
plt.title('Monthly Total Sales (2014)')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [30]: # Monthly total sales
# Identifying high and low sales months
max_sales_month = monthly_sales.loc[monthly_sales['Total Sales'].idxmax()]
min_sales_month = monthly_sales.loc[monthly_sales['Total Sales'].idxmin()]
print(f'Uniquely Sales Month:')
print(max_sales_month)
print(min_sales_month)
print(min_sales_month)

Highest Sales Month:
Year-Month    2014-01
Total Sales    503479.4488
Name: 0, dtype: object

Minimum sales month:
Year-Month    2014-05
Total Sales    228989.6496
Name: 4, dtype: object
```

- Based on the 2014 sales trends, the following marketing strategy is proposed for 2015:

January recorded the highest sales, around 5.06 million BDT (5,06,596), indicating increased consumer demand during the New Year season. The second highest sales occurred in December, approximately 4.09 million BDT (4,09,163), likely due to Christmas and year-end shopping. The third highest sales were in March, around 4.04 million BDT (4,04,633).

On the other hand, the lowest sales were observed in May, June, and November. To boost sales during these months, various campaigns and customer discounts can be introduced. Specifically, a Winter Discount Campaign in November could significantly increase sales during that period.

Task 4: Discount Strategy

- Plot average discount by Order Priority (bar plot).
- Propose a discount adjustment based on discount-sales correlation (use correlation matrix or scatter plot). Justify in 2-3 sentences.

```
In [20]: # Average Discount by Order Priority
avg_discount_by_priority = df.groupby('Order Priority')['Discount'].mean().sort_values(ascending=False)
print(avg_discount_by_priority)

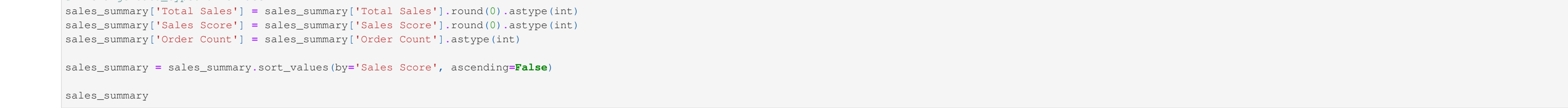
Order Priority
Low          5.138095
High         5.064148
Critical     4.828442
Medium      4.814402
Not Specified 4.615730
Name: Discount, dtype: float64
```

```
In [26]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.barplot(x=avg_discount_by_priority.index, y=avg_discount_by_priority.values, palette='viridis')

for i, value in enumerate(avg_discount_by_priority.values):
    plt.text(value - 0.002, i, f'{value:.2%}', va='center', ha='right', color='black', fontsize=10, fontweight='bold')
```

```
plt.title('Average Discount by Order Priority')
plt.xlabel('Average Discount (%)')
plt.ylabel('Order Priority')
plt.tight_layout()
plt.show()
```

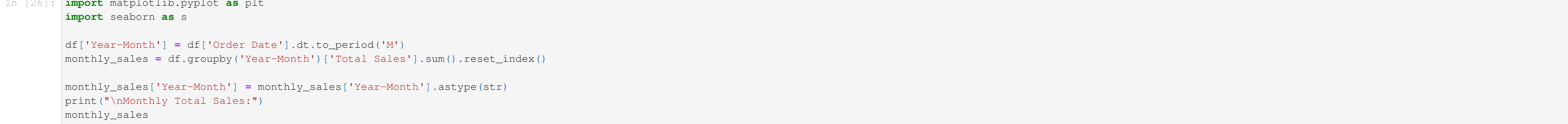


```
In [28]: # Correlation between Discount and Total Sales
correlation = df[['Discount', 'Total Sales']].corr()
print("Correlation Matrix:")
print(correlation)

# Scatter Plot
plt.figure(figsize=(8, 6))
sns.scatterplot(x='Discount', y='Total Sales', data=df, alpha=0.6, color='darkgreen')
plt.title('Discount vs Total Sales')
plt.xlabel('Discount')
plt.ylabel('Total Sales')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Correlation Matrix:

	Discount	Total Sales
Discount	1.00000	0.00000
Total Sales	-0.03615	1.00000



```
In [30]: # Calculate average discount by Order Priority
avg_discount_by_priority = df.groupby('Order Priority')['Discount'].mean().sort_values(ascending=False)

# Correlation value
correlation = df[['Discount', 'Total Sales']].corr().loc['Discount', 'Total Sales']

# Recommendation Message
print(f"Discount Strategy Recommendation:")
print(f"The correlation between Discount and Total Sales is {correlation:.4f}."
      f"Instead, target higher discounts around {avg_discount_by_priority['Low']:.2%} or {avg_discount_by_priority['Not Specified']:.2%} * "
      f"Low and 'Not Specified' priority orders to encourage purchases."
      f"Maintain lower discounts on 'Critical' and 'High' priority orders to preserve profit margins.")
```

Discount Strategy Recommendation:

The correlation between Discount and Total Sales is -0.0361. Given the weak correlation, increasing discounts across all orders may not significantly boost sales. Instead, target higher discounts around 5.14% to 4.62% on 'Low' and 'Not Specified' priority orders to encourage purchases. Maintain lower discounts on 'Critical' and 'High' priority orders to preserve profit margins.

Task 5: Shipping Analysis

- Visualize orders by Ship Mode (bar/line chart).
- Recommend one shipping cost reduction strategy. Justify in 2-3 sentences, considering Order Priority or Product Container.

```
In [21]: ship_mode_counts = df['Ship Mode'].value_counts()
ship_mode_counts
```

```
Out[21]:
```

Regular Air	1586
Delivery Truck	303
Express Air	244
Name: count, dtype: int64	

```
In [21]: # Pie chart
plt.figure(figsize=(6, 5))
colors = sns.color_palette('coolwarm', len(ship_mode_counts))
plt.pie(ship_mode_counts, labels=ship_mode_counts.index, autopct='%1.1f%%', colors=colors, startangle=140)
plt.title('Order Distribution by Ship Mode')
plt.tight_layout()
plt.show()
```



```
In [22]: # Average Shipping Cost by Ship Mode and Order Priority
shipping_cost = df.groupby(['Ship Mode', 'Order Priority'])['Shipping Amount'].mean().unstack()

# Also check shipping cost by Product Container to see any pattern
shipping_cost = df.groupby('Product Container')['Shipping Amount'].mean().sort_values(ascending=False)
shipping_cost, container_cost
```

```
Out[22]:
```

	Order Priority	Critical	High	Low	Medium	Not Specified
Delivery Truck	46.277188	44.289355	43.803607	44.022500	42.341854	
Express Air	9.728300	8.335680	8.677259	10.216964	9.206909	
Regular Air	6.257951	8.227401	8.252942	8.151667	7.601707	
Product Container						
Jumbo Box	50.225809					
Large Box	38.532073					
Medium Box	32.000357					
Small Box	13.999709					
Small Rack	7.395459					
Small Rack	4.630881					
Small Rack	2.732623					
Name: Shipping Amount, dtype: float64						

the most expensive shipping method is Delivery Truck, especially when paired with large containers like Jumbo Box and Jumbo Drum. To reduce shipping costs, it is recommended.

Task 6: Recommendations

Write a 100-150 word summary:

- Key findings from Tasks 2-5.
- One business recommendation.

